

Executive Summary

The **CareerPilot** Flask/SQLite job-tracker can be enhanced to match many features of commercial “paid” academic projects while still looking like a student effort. Paid project vendors typically include polished UIs (often Bootstrap-based), authentication, search/filter functions, visualizations, and extensive documentation (UML diagrams, reports, PPTs) ¹ ². To emulate this, we propose targeted student-implementable improvements: use Bootstrap for a modern responsive UI, add hashed-password login (via Flask-Login), implement search and status-filtering in SQL, show simple analytics (status counts), and harden security (use parameterized queries) ³ ⁴. In parallel, prepare concise documentation (ER diagram, report abstract, key screenshots) in clear student language.

This report gives: a **comparison table** of typical paid-project features versus our enhancements; a **prioritized improvement list** (UI/UX, backend, DB, security, docs) with estimated effort; **code snippets** (login hashing, search query, etc.); sample **report/viva text** phrased in a student’s voice; a **final submission checklist**; advice on avoiding plagiarism; and recommended resources and example project sites for reference. Official sources (Flask, SQLite, Bootstrap docs, security best-practices) guide our technical suggestions. For example, Flask’s documentation shows `generate_password_hash` for secure password storage ⁵, and SQLite docs warn to use `?` placeholders in SQL to prevent injection ⁴. By following these best practices and keeping explanations simple, the project will be robust and presentable without any “copied” feel.

Paid-Project Features vs Student-Enhancements

Commercial project packages often advertise many bells-and-whistles. The table below compares common paid-project features with practical enhancements a student can implement. Each enhancement is realistic for a small Flask/SQLite app:

Feature / Aspect	Typical Paid-Project Package	Student Version (Enhancement)
Authentication	Full login system with Flask-Login , roles, “Remember me” ⁶ ¹ . Passwords hashed (often with bcrypt or Werkzeug).	Basic login/logout using Flask-Login or session cookies. Hash passwords with Werkzeug’s <code>generate_password_hash</code> ⁵ . One user role (student only).
UI/UX Design	Polished responsive design (Bootstrap/ MDBBootstrap/ Material), icons, animations, interactive tables.	Use Bootstrap 5 (free) for layout, forms, navbars, tables. Add <code>table-striped table-hover</code> classes and wrap with <code>.table-responsive</code> ⁷ for mobile-friendly tables. Use simple icons (FontAwesome).

Feature / Aspect	Typical Paid-Project Package	Student Version (Enhancement)
Search & Filtering	Multi-field AJAX search, dropdown filters, real-time updates.	Implement basic search using SQL <code>LIKE</code> or <code>.filter()</code> on job title/company/date. E.g. <code>Job.query.filter(Job.title.ilike(f"%{q}%"))</code> . Add a dropdown or checkboxes to filter by status.
Data Analytics	Charts (e.g. Chart.js) showing counts by status, dates, etc.	Show summary counts (e.g. "3 Applied, 1 Interview, 0 Offered"). These can be plain Bootstrap badges or progress bars. (Optional: embed a simple Chart.js for status pie or bar).
Data Operations (CRUD)	Admin + user panels: full Create/Edit/Delete for jobs and users ⁸ . File uploads, email notifications.	Implement basic Add/Edit/Delete routes for jobs. Use Flask routes and HTML forms for each. (No file uploads needed for simple job tracker).
Responsive Layout	Fully responsive pages at all breakpoints with custom CSS.	Use Bootstrap's grid (<code>container</code> , <code>row</code> , <code>col</code>) so pages adapt. E.g. <code><div class="container"></code> with <code>.col-md-6</code> for forms. This ensures usability on mobile.
Security	CSRF protection, strong password hashing (bcrypt), session fixation protection.	Set <code>app.secret_key</code> to a random value (Flask-Login requires this) ³ . Use Werkzeug hashing (<code>generate_password_hash</code> / <code>check_password_hash</code>) ⁵ ⁹ . Use parameterized queries or ORM to avoid SQL injection ⁴ . (CSRF tokens via Flask-WTF are optional but can be mentioned.)
Documentation	Full Project Report, PPT, UML/ER/DFD diagrams included ¹ ² .	Provide a concise report (1–2 pages) describing design and features. Include an ER diagram (see example below) and sample screen-shots . Optionally list tables/fields. PPT can be minimal (if required).
Code Quality	Well-structured (blueprints/modules), requirements.txt, README, comments.	Organize code simply: e.g. put routes in <code>app.py</code> , HTML in <code>templates/</code> , CSS in <code>static/</code> . Write a <code>requirements.txt</code> . Add comments to explain key logic. Rename variables from tutorial examples to be project-specific.
Additional Perks	UML diagrams, video demo, customer support, "100% working" guarantee ² .	Skip marketing fluff. Instead, highlight what you <i>actually</i> did: tested forms, data persistence, mobile view. You may note "based on Flask official docs" or cite a reference in comments if inspired by an example. Include a simple ER diagram and maybe one use-case diagram if class requires.

The above table shows that most paid features can be approximated by lightweight versions: e.g. Bootstrap for UI, ORMs or parameterized queries for security, and basic reporting features. Keep explanations honest: e.g. "Used Flask session cookies" rather than claiming full JWT authentication.

Prioritized Improvements (with Effort Estimates)

Below is a prioritized list of enhancements, grouped by category. Each item includes an *estimated effort/time* (very roughly) for a motivated student. Prioritize higher-impact items first.

• UI/UX

- *Add Bootstrap styling (2–4 hours)*: Integrate Bootstrap via CDN. Convert forms and tables to use Bootstrap classes. Ensure tables are within `<div class="table-responsive">` ⁷ so columns scroll on mobile. Use a Navbar and container layouts. This instantly looks much more professional.
- *Improve Form Layout (1–2h)*: Use Bootstrap grid or form classes to align labels/inputs. Add placeholder text and `required` attributes for basic validation. Use `form-control` classes on inputs/buttons.
- *Status badges/indicators (1h)*: Display job status with colored badges (e.g. `<span class="badge bg-primary">Applied</span>` ⁷) so it stands out.

• **Effort total UI:** ~1 day.

• Backend and Functionality

- *Login & Authentication (2–4h)*: Use Flask-Login (or manual session cookies) to protect the dashboard. Hash passwords with Werkzeug (see code below) ⁵. Ensure `app.secret_key` is set (flask docs warn this is required) ³. Protect routes with `@login_required`. (Source: Flask-Login docs say sessions need a secret key ³.)
- *Search and Filter (1–2h)*: Add a search form on the jobs list page. In the route, get the query and filter: e.g. `jobs = Job.query.filter(Job.title.ilike(f"%{q}%") | Job.company.ilike(f"%{q}%")).all()`. To avoid SQL injection, always use ORM or parameterized statements (see security below). Optionally, add a dropdown to filter by `status`.
- *Edit/Delete Flows (1–2h)*: Provide "Edit" and "Delete" links/buttons for each job row. These go to routes like `@app.route('/delete/<int:job_id>')` and remove the entry after confirming. Add `flash("Job deleted", "info")` messages. (Flask's `flash()` function is documented in [25] as well.)
- *Status Counts/Analytics (1h)*: In the dashboard, query counts by status: e.g. `applied = Job.query.filter_by(status='Applied').count()`. Display these numbers at top (e.g. "Applied: X, Interview: Y, Offered: Z") or use a simple chart. Bootstrap progress bars or charts.js can show this.

• **Effort total Backend:** ~1–2 days.

• Database

- *Data Model Review (1h)*: Ensure each table has a primary key and foreign key. For a single-user app, maybe only a `jobs` table is needed with fields `(id PK, title, company, date_applied,`

`status, comments)` . If supporting multiple users, add a `user_id` FK. A simple ER diagram is shown below.

- *Migrations/Schemas (optional, 1–2h)*: If using raw SQLite, create a `schema.sql` and initialize DB automatically (Flask tutorial suggests this). If using SQLAlchemy, ensure tables are created in `app.py` (e.g. `db.create_all()`). Provide the DB file in submission or instructions to generate it.
- *Data Validation (1h)*: Add basic validation on form inputs (e.g. date format). This can be as simple as HTML `type="date"` or checking in the route before inserting.

• **Effort total DB**: ~half a day.

• Security

- *Password Hashing (1h)*: Store only hashed passwords. Example: `user.pw_hash = generate_password_hash(input_pwd)` ⁵ , and check with `check_password_hash` ¹⁰ . Do **not** keep plaintext. This meets any basic requirement for authentication.
- *Parameterized Queries (0.5h)*: Avoid building SQL with string concatenation. Use `cursor.execute(sql, params)` with `?` placeholders in raw SQL, or ORM filters (which auto-parameterize). The Python docs explicitly warn to use `?` to prevent SQL injection ⁴ . For example:

```
query = "SELECT * FROM jobs WHERE title LIKE ?"
cursor.execute(query, (f"%{search_term}%",))
```

This prevents malicious input from breaking the query ⁴ .

- *Session Security (0.5h)*: Ensure Flask `SECRET_KEY` is set to a random string (not “secret”). This secures session cookies ³ . Optionally, regenerate session id on login for extra safety.

• **Effort total Security**: ~1–2 hours.

• Documentation & Report

- *ER Diagram (1h)*: Draw a simple Entity-Relationship diagram for your data model. For example, **User** (id, username, password) and **Job** (id, title, company, date_applied, status, user_id). We include a sample below.
- *Screenshots (0.5h)*: Take clear screenshots of the running app (login page, dashboard with job list). These are often expected in reports.
- *Report Write-up (2–3h)*: Update your project report to mention each new feature. Use student-style language: e.g. “We used Bootstrap for the UI, making the site mobile-friendly.” Keep it factual and concise. Emphasize what you did, not what you didn’t.
- *Viva Q&A Preparation (1h)*: Write a few sample Q&A. For instance, “**Q**: How did you implement search? **A**: I added a form that sends a query string to the Flask route; in Python I used an SQL `LIKE` filter to match jobs. I also made sure to use `?` placeholders to avoid injection ⁴ .” Such answers show you understand your code.

• **Effort total Documentation**: ~1 day.

In summary, **front-end work (Bootstrap/UX) and refactoring login/search** are highest impact and moderate effort (1–2 days). Security fixes (hashing, query param) are quick but important. Preparing the write-up and diagrams may take an additional day. Overall, the project can reach a much more polished level with a few intensive days of work.

Implementation Snippets and Illustrations

Below are concrete examples of the key improvements. (These are illustrative; adapt to your code structure.)

1. User Login and Password Hashing

```
from werkzeug.security import generate_password_hash, check_password_hash
from flask_login import LoginManager, login_user, login_required, logout_user,
current_user

# Configure Flask-Login
login_manager = LoginManager()
login_manager.init_app(app)
app.secret_key = 'a-very-secret-key' # use a random generator in practice 3

# User model (example using Flask-SQLAlchemy)
class User(db.Model, UserMixin):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    password_hash = db.Column(db.String(128), nullable=False)

    def check_password(self, pwd):
        return check_password_hash(self.password_hash, pwd) # official doc 9

# Registration: hash the password
@app.route('/register', methods=['GET', 'POST'])
def register():
    if form.validate_on_submit():
        username = form.username.data
        raw_pw = form.password.data
        hashed_pw = generate_password_hash(raw_pw) # official Werkzeug API 5
        user = User(username=username, password_hash=hashed_pw)
        db.session.add(user)
        db.session.commit()
        flash("Registration successful", "success")
        return redirect(url_for('login'))
    return render_template('register.html', form=form)

# Login: check the hash
@app.route('/login', methods=['GET', 'POST'])
def login():
```

```

if form.validate_on_submit():
    user = User.query.filter_by(username=form.username.data).first()
    if user and check_password_hash(user.password_hash, form.password.data):
        login_user(user) # now the user is logged in via Flask-Login
        flash("Logged in", "success")
        return redirect(url_for('dashboard'))
    else:
        flash("Invalid username or password", "danger")
return render_template('login.html', form=form)

```

Explanation: We import `generate_password_hash` and `check_password_hash` from Werkzeug's security module ⁵ ⁹. On registration, we store the hashed password. On login, we verify it. The use of `login_user` and setting `app.secret_key` comes from Flask-Login's docs ³. This prevents storing plaintext passwords and secures sessions.

2. Search and Filter Query

To add a basic search bar (e.g. in `dashboard.html`):

```

<form method="get" action="{{ url_for('dashboard') }}">
  <input type="text" name="q" class="form-control" placeholder="Search jobs...">
  <button type="submit" class="btn btn-primary">Search</button>
</form>

```

And in the route (`app.py`):

```

@app.route('/dashboard')
@login_required
def dashboard():
    q = request.args.get('q', '')
    if q:
        # Safe query using parameter binding (SQLAlchemy handles it) 4
        jobs = Job.query.filter(
            (Job.title.ilike(f"%{q}%")) |
            (Job.company.ilike(f"%{q}%"))
        ).all()
    else:
        jobs = Job.query.all()
    return render_template('dashboard.html', jobs=jobs, query=q)

```

If using raw SQLite, do:

```

conn = sqlite3.connect('jobs.db')
cur = conn.cursor()

```

```
cur.execute("SELECT * FROM jobs WHERE title LIKE ? OR company LIKE ?", (f"%{q}%%", f"%{q}%")) # qmark placeholders 4
results = cur.fetchall()
```

Explanation: We use `LIKE` with `%` for substring search. Importantly, **we use query parameters (placeholders)**, so user input is treated as data, not SQL code ⁴. This avoids injection. The above uses SQLAlchemy ORM, which internally parameterizes the query. Alternatively, SQLite's `execute(..., params)` is used with `?` placeholders, as recommended in the Python docs ⁴.

3. Status Counts / Simple Analytics

In the dashboard route we can calculate counts:

```
statuses = ['Applied', 'Interview', 'Offered', 'Rejected']
counts = {status: Job.query.filter_by(status=status).count() for status in statuses}
```

Then pass `counts` to the template, and display in Bootstrap cards:

```
<div class="row mb-3">
  {% for status, count in counts.items() %}
  <div class="col-md-3">
    <div class="card text-center">
      <div class="card-body">
        <h5 class="card-title">{{ status }}</h5>
        <p class="card-text display-4">{{ count }}</p>
      </div>
    </div>
  </div>
  {% endfor %}
</div>
```

Explanation: This shows a simple dashboard summary without requiring a chart library. If desired, one could use Chart.js to turn these counts into a bar or pie chart. Even this basic count display looks like a dashboard widget.

4. Edit/Delete Flow & Flash Messages

For each job in the job list table, add “Edit” and “Delete” buttons:

```
<table class="table table-striped table-hover">
<thead><tr><th>Title</th><th>Company</th><th>Status</th><th>Actions</th></tr></thead>
<tbody>
```

```

{% for job in jobs %}
<tr>
  <td>{{ job.title }}</td>
  <td>{{ job.company }}</td>
  <td><span class="badge bg-info">{{ job.status }}</span></td>
  <td>
    <a href="{{ url_for('edit_job', job_id=job.id) }}" class="btn btn-sm btn-secondary">Edit</a>
    <a href="{{ url_for('delete_job', job_id=job.id) }}" class="btn btn-sm btn-danger"
      onclick="return confirm('Delete this job?');">Delete</a>
  </td>
</tr>
</tbody>
</table>

```

And example delete route:

```

@app.route('/delete/<int:job_id>')
@login_required
def delete_job(job_id):
    job = Job.query.get_or_404(job_id)
    db.session.delete(job)
    db.session.commit()
    flash("Job deleted successfully.", "info") # simple flash message
    return redirect(url_for('dashboard'))

```

Explanation: We use Bootstrap classes (`btn btn-sm`, `badge`) for a cleaner UI. The `flash()` function (see Flask-Login tutorial) shows a temporary message after actions ¹¹. Using `get_or_404` ensures a safe lookup. Always require login on these routes.

5. Responsive Layout Example

Wrap your content in a Bootstrap container, e.g.:

```

<nav class="navbar navbar-expand-lg navbar-light bg-light">
  <a class="navbar-brand" href="#">CareerPilot</a>
  <!-- navbar toggler for mobile -->
</nav>
<div class="container mt-4">
  <!-- content here -->
</div>

```

For tables, use the responsive wrapper as shown:

```
<div class="table-responsive">
  <table class="table">
    <!-- table rows -->
  </table>
</div>
```

According to Bootstrap docs: “Make any table responsive across all viewports by wrapping a `.table` with `.table-responsive`” ⁷. This ensures horizontal scrolling on small screens.

6. ER Diagram (Data Model)

```
erDiagram
    USER ||--o{ JOB : has
    USER {
        int id PK "Primary key"
        string username
        string password_hash
    }
    JOB {
        int id PK "Primary key"
        string title
        string company
        date applied_date
        string status
        int user_id FK "Foreign key to User.id"
    }
```

Explanation: A simple ER diagram showing that each **User** can have many **Job** entries. We store `password_hash` instead of plaintext (Werkzeug). For a single-user app, you might omit the User table.

7. Page Flow Diagram

```
flowchart LR
    A[Login Page] --> B[Dashboard]
    B --> C[Add Job Form]
    B --> D[Job List / View Jobs]
    B --> E[Search/Filter Jobs]
    D --> F[Edit Job Form]
    D --> G[Delete Confirmation]
    C --> B
    F --> B
    G --> B
```

Explanation: This flowchart outlines the main user interactions: login, viewing the dashboard with job list, adding a job, editing, deleting, and searching. It shows that after each operation (add/edit/delete) the user returns to the dashboard.

Sample Report and Viva Wording

Below are excerpts written in a student-friendly style, illustrating how to describe and discuss the changes:

Project Implementation (UI/UX):

"We improved the web interface by adding the Bootstrap framework (via CDN). The dashboard now uses a responsive grid and table, so it looks good on desktop and mobile ⁷. For example, the job list is inside a `.table-responsive` wrapper, enabling horizontal scroll on small screens ⁷. We also used Bootstrap classes (`table-striped`, `btn btn-primary`, etc.) to make the layout clean and consistent. This approach is common in many projects and helps present the data more clearly."

Project Implementation (Features):

"We implemented a search bar to filter jobs by title or company. In the Flask route, we used an `LIKE` query with `%` wildcards. Importantly, we passed user input as a parameter to the query instead of inserting it into the SQL string ⁴. This prevents SQL injection and is best practice according to the Python docs ⁴. We also added status summary badges on the dashboard that count how many jobs are in each stage (e.g. "Applied: 3"). These are computed with simple `COUNT()` queries in Python."

Project Implementation (Security):

"We improved security by hashing user passwords with Werkzeug. In the registration code, we call `generate_password_hash(password)` ⁵; this stores a salted hash in the database. During login, we check the hash with `check_password_hash` ⁹. We also ensured `app.secret_key` is set (Flask requires this for session cookies) ³. These steps follow official Flask recommendations for authentication ³ ⁵."

Viva Q&A Samples:

- Q: "What libraries or frameworks did you use to improve your project?"

A: "I used **Bootstrap 5** for styling and layout, because it's free and easy to apply. It made forms and tables look better with minimal coding. I also used **Flask-Login** for managing user sessions, and Werkzeug's security module for password hashing ⁵. These are standard tools recommended in the Flask docs."

- Q: "How did you implement the search feature?"

A: "I added a search form that sends a query parameter to the Flask route. In the Python code, I filter the jobs using SQLAlchemy's `filter(... ilike(...))`, which safely inserts the search term. This avoids SQL injection. For example, using `?` placeholders in raw sqlite is advised to prevent injection ⁴."

- Q: "What changes did you make to the database?"

A: "I ensured our `jobs` table has a primary key and added a foreign key `user_id` if multiple users are supported. I also added an index on status to speed up counting by status (optional). The SQLite docs show how to connect and create tables safely ¹²."

- Q: "How have you documented your project?"

A: "I wrote a concise report including the problem statement, my design (including the ER diagram above), and the features we added. I included screenshots of key pages (login and dashboard). I also prepared Q&A (like this) to be ready for viva questions. Everything is explained in straightforward terms."

These examples show how to phrase changes in first person, focusing on *what we did* and *why*. Avoid saying "AI helped" or copying marketing lines like "100% working" ²; instead use terms like "Bootstrap styling," "password hashing," and cite credible sources if needed.

Submission Checklist

Before final submission, ensure all the following are included and correct:

- **Screenshots:** Capture and include images of:
 - Login page (before/after if changed).
 - Dashboard or job list (showing at least 3 jobs, with status badges visible).
 - Add/Edit Job form.
 - Any special feature (e.g. search in action, status summary section).
(Use only actual screenshots from your app; label them in the report.)
- **File Structure:** Verify your project has:
 - A clear folder organization (e.g. `app.py` or `main.py`, `templates/`, `static/` (with CSS/JS if any), `requirements.txt`).
 - If using a package, ensure `__init__.py` and modules are included.
 - An SQLite file (e.g. `career_pilot.db`) or instructions to create it.
 - **Requirements:** Provide a `requirements.txt` listing needed packages (Flask, Flask-Login, etc.). Alternatively, specify in instructions (e.g. `pip install flask flask-login` etc.).
 - **Setup Instructions:** In a README or report, include:
 - How to set `FLASK_APP` if needed (e.g. `export FLASK_APP=app.py`).
 - How to initialize the DB (e.g. run a provided `schema.sql` or a setup script).
 - How to run the app (e.g. `flask run` or `python app.py`).
 - Mention any default credentials (e.g. "A sample user: admin / admin").
 - **Report/Documentation:** Include:
 - The ER diagram above (can redraw by hand or using tools).
 - A brief **abstract/synopsis** explaining the project's purpose.
 - A **system requirements** section (e.g. Python3, pip).
 - A **feature list** (can mirror the comparison table row names).
 - Any UML/use-case diagrams if available (recommended, but keep them simple).
 - Screenshots (as noted).
 - A **conclusion** or **future work** note (optional).
 - **Code Comments:** Ensure any borrowed code is commented with its source (or rewritten in your own style). Remove any "demo" or "example" markers. Avoid leaving commented-out test code.
 - **PPT (if required):** If a presentation is requested, make a few slides: Title, objective, tech stack, features (with screenshots), and conclusion. Keep text minimal and speak to screenshots/images.

- **Commands to Run:** For clarity, you might include a section in the README or report like:

```
pip install -r requirements.txt
flask run
```

or

```
python app.py
```

as applicable.

Following this checklist helps avoid “missing file” issues and ensures the grader can easily verify your work.

Avoiding Plagiarism and “Copied” Fingerprints

When upgrading to a paid-project level, be careful **not** to copy text or designs directly from internet sources. Some common pitfalls:

- **Marketing Language:** Sites often use phrases like “100% working, bug-free” ², “Full source code and report included” ¹, or “easy to customize”. Avoid parroting these in your report – instead say what *you* did. (For instance, do **not** claim “bug-free” — just say “tested functionality”).
- **Exact Feature Lists:** Paid sites list many features (cart systems, notifications, etc.). Only claim features you implemented. Don't copy a feature list from a site unless you actually added those features.
- **Report/Documentation Format:** Look at examples (like GitHub repos or official guides), but write the synopsis and conclusion in your own words. You can read how others describe, but explain as **you** would: “In our project, we implemented X and Y...”.
- **Code Snippets:** If you adapt an example (e.g. from Flask docs or StackOverflow), do understand and possibly rewrite it so it fits your variable names. Don't include large chunks of copied code without credit. A small snippet (like using `generate_password_hash`) is fine, but make sure you understand each line.
- **Diagrams:** If you use online generators (draw.io, Lucidchart), avoid using copyrighted images. Instead, draw or use tools to make your own ER/use-case diagrams.
- **Citing Sources:** If you find a helpful solution (e.g. on StackOverflow), you can mention it in comments. In your report, it's okay to say “the code is based on [Flask docs] or [TutorialX]” with a proper reference. This shows honesty and gives credit.
- **Fiverr or Paid Copywriting:** If you see Fiverr gigs (like the one above), notice they often mention supporting viva or customization ¹³ ¹⁴. While such offerings are **for students**, copying their advert text or Q&A is plagiarism. Instead, prepare your own answers as above.

In short: use official docs and tutorials for learning (cite them if a point is non-obvious), but never copy text verbatim into your report or comments. Think “how would **I** explain this feature”. This keeps the project authentically **your** work.

Recommended Resources and Example Sites

For guidance and inspiration, consult official documentation and reputable examples:

- **Flask Documentation (Pallets)** – The official guide covers routing, templates, sessions, and security ³. See especially *Flask-Login* usage for sessions ³ and *Werkzeug* security docs for password hashing ⁵ ⁹.
- **SQLite Python docs** – The [sqlite3 module documentation](#) explains how to connect and use [?] placeholders ¹² ⁴. It explicitly shows using `cursor.execute("...?", (param,))` to avoid SQL injection ⁴.
- **Bootstrap Docs** – The official Bootstrap site describes responsive tables and forms ⁷. Use the examples for tables (`.table`, `.table-responsive`) and navbars.
- **Flask Mega-Tutorial (Miguel Grinberg)** – Free online tutorial with examples of login, databases, etc. (Not directly cited here, but a useful learning resource).
- **StackOverflow / GeeksforGeeks** – For specific code examples (e.g., “Flask search query”, “Flask flash messages”). Always adapt from these, and cite accordingly if you copy an idea.

Example project sites (for feature ideas) – (These are paid or community sites often used by students. Review them to see common features, but do not copy content verbatim.)

1. **Aislyn Technologies (aislyn.in)** – A vendor listing many Flask/BCA projects ¹. They advertise “Full Source Code, PPT, Abstract, Report” etc. ¹. Good for checking typical features (but use the phrasing in your words).
2. **Kashipara (kashipara.com)** – A community site with hundreds of project downloads. See how they list features (e.g. “Add/Edit/Delete” operations) ⁸. This gives ideas of necessary CRUD functionality.
3. **ProgrammingFever (programmingfever.com)** – An online “shop” for student projects. Pages show titles like “*Doctor Appointment Management System (with Source Code and Report)*” ¹⁵. Pricing pages reveal what’s included (code + PPT/report).
4. **Fiverr / Freelancer** – On Fiverr you can find gigs selling mini-projects. For example, one gig offers “Python project + report outline + PPT + viva Q&A” ¹³ ¹⁴. This highlights common deliverables. However, remember these are professional offers – use them to understand scope, not to copy text.
5. **ProjectWorlds / YouTube demos** – Sites like ProjectWorlds list free code or walkthroughs (e.g. “*Flask ToDo App*”). Watching a demo can give UI ideas (layout, navigation). Official Flask tutorials and GitHub repositories (search “Flask job tracker”) also provide patterns.

By combining official docs (for correctness) with observing academic project sites (for scope), you can safely advance your project. Always note the source of any external idea and rephrase it in your report.

Sources: We relied on Flask and Python official documentation for technical guidance ³ ⁴. The paid-project sites cited (Aislyn, ProgrammingFever, Kashipara) illustrate typical academic project offerings ¹ ¹⁵ ⁸. Security best practices come from the Snyk blog and Python docs ¹⁶ ⁴. These references helped shape the improvements listed above.

¹ Python Flask Projects for Final Year Students | Aislyn Tech
<https://aislyn.in/blog/python-flask-projects>

2 Portal Of Insurance Management System Project in Python with Source Code and Report [download] - kashipara

<https://www.kashipara.com/project/python/13142/insurance-management-system-using-python-project-source-code>

3 6 11 Flask-Login 0.7.0 documentation

<https://flask-login.readthedocs.io/en/latest/>

4 12 sqlite3 — DB-API 2.0 interface for SQLite databases — Python 3.14.4 documentation

<https://docs.python.org/3/library/sqlite3.html>

5 werkzeug.generate_password_hash — Flask API

https://tedboy.github.io/flask/generated/werkzeug.generate_password_hash.html

7 Tables · Bootstrap

<https://getbootstrap.com/docs/4.0/content/tables/>

8 Product Management System Using Python Project in Python with Source Code and Report [download] - kashipara

<https://www.kashipara.com/project/python/13233/product-management-system-using-python-python-project-source-code>

9 10 werkzeug.check_password_hash — Flask API

https://tedboy.github.io/flask/generated/werkzeug.check_password_hash.html

13 14 Provide a python student record management mini project for students by Annu_vid | Fiverr

https://www.fiverr.com/annu_vid/provide-a-python-student-record-management-mini-project-for-students

15 Shop – ProgrammingFever

<https://programmingfever.com/shop/>

16 How to secure Python Flask applications | Snyk

<https://snyk.io/blog/secure-python-flask-applications/>